

Agentic RAG System — Full Code Walkthrough

Explained for Ishwar

August 2026

What Are We Building?

We are building an **Agentic RAG (Retrieval-Augmented Generation) chatbot backend**.

In plain words: this is a question-answering system that doesn't just blindly search a database and hope for the best. Instead, it acts like a small team of workers (called "nodes") who each check each other's work before handing over the final answer. That's why it's called "agentic" — the system makes decisions on its own about what to do next, instead of following one fixed straight-line path.

Here is the plain-language story of what happens when a user asks a question:

1. A **router** first reads the question and decides: "Is this something I have documents about (agents, prompt engineering, adversarial attacks on LLMs), or should I just search the internet for it?"
2. If it belongs in our own document collection, we **retrieve** matching chunks of text from a local vector database (Chroma).
3. Those retrieved chunks are **graded** one by one — an LLM checks "is this chunk actually relevant to the question?" Irrelevant ones get thrown away.
4. If too many chunks got thrown away, the system decides it doesn't have enough good material, so it triggers a **web search** (using Tavily) to fetch fresh information from the internet instead.
5. Once we have good material (from our documents and/or the web), we **generate** an answer using an LLM.
6. Before showing the answer to the user, two more checks run:
 - **Hallucination check** — is the answer actually based on the retrieved facts, or did the LLM make something up?
 - **Answer-usefulness check** — does the answer actually address the question that was asked?
7. Depending on the result of these checks, the system either shows the final answer, tries generating again, or falls back to a web search and tries again.

This entire decision flow (route → retrieve → grade → maybe web-search → generate → grade the generation → maybe retry) is built using **LangGraph**, a library that lets you draw this like a flowchart/graph, where each box is a Python function ("node") and each arrow is a rule about what happens next ("edge"). The `graph.png` file in the repo is literally a picture of this flowchart.

How the Pieces Fit Together (Integration Map)

- `ingestion.py` — runs once, offline. It downloads 3 blog posts, cuts them into small chunks, converts each chunk into numbers (embeddings), and stores them in a local vector database called Chroma. This builds the "knowledge base" the retriever will search later.
- `graph/state.py` — defines the shared notebook (`GraphState`) that every node reads from and writes to as the question flows through the system.
- `graph/consts.py` — just gives short, typo-proof names (constants) to each node, so the rest of the code doesn't use raw strings everywhere.
- `graph/nodes/` — the actual workers: `retrieve`, `grade_documents`, `generate`, `web_search`. Each one is a plain Python function that takes the current state and returns updates to it.
- `graph/chains/` — these are the "LLM calls" that the nodes lean on: `router` (decide vectorstore vs web), `retrieval_grader` (is this chunk relevant?), `generation` (write the answer), `hallucination_grader` (is the answer grounded in facts?), `answer_grader` (does the answer address the question?). Each chain = a prompt + an LLM + (sometimes) a structured output shape, wired together with LangChain's `|` (pipe) syntax.
- `graph/graph.py` — the wiring diagram. It creates the graph, adds all the nodes, and adds the arrows (edges) between them, including the "if/else" style conditional arrows. Finally, it compiles everything into a runnable `app` and saves a picture of the flow to `graph.png`.
- `main.py` — the entry point. Loading this file loads everything above it and actually runs one real question through the whole system, then prints the final result.

Now let's go through every file, line by line, in plain language.

`ingestion.py` — Building the Knowledge Base

```
from dotenv import load_dotenv
from langchain_chroma import Chroma
from langchain_community.document_loaders import WebBaseLoader
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

load_dotenv()

urls = [
```

```

    "https://lilianweng.github.io/posts/2023-06-23-agent/",
    "https://lilianweng.github.io/posts/2023-03-15-prompt-engineering/",
    "https://lilianweng.github.io/posts/2023-10-25-adv-attack-llm/",
]

docs = [WebBaseLoader(url).load() for url in urls]
docs_list = [item for sublist in docs for item in sublist]

text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    chunk_size=250, chunk_overlap=0
)

doc_splits = text_splitter.split_documents(docs_list)

embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")

vectorstore = Chroma.from_documents(
    documents=doc_splits,
    collection_name="rag-chroma",
    embedding=embeddings,
    persist_directory="./.chroma",
)

retriever = Chroma(
    collection_name="rag-chroma",
    persist_directory="./.chroma",
    embedding_function=embeddings,
).as_retriever()

```

Imports — why each one is here:

- `from dotenv import load_dotenv` — This lets the code read secret values (like API keys) from a hidden `.env` file on your computer instead of typing them directly into the code. Nobody wants API keys sitting in plain text in a file that gets uploaded to GitHub.
- `from langchain_chroma import Chroma` — Chroma is a lightweight local database made specifically for storing “embeddings” (numbers that represent meaning) and searching through them fast. We import it so we can build and later search our knowledge base.
- `from langchain_community.document_loaders import WebBaseLoader` — This is a ready-made tool that knows how to open a webpage URL, strip out the HTML tags, and give us back clean readable text. Saves us from writing our own web scraper.
- `from langchain_huggingface import HuggingFaceEmbeddings` — This gives us access to a free, local AI model whose only job is to turn text into a list of numbers (a “vector”) that captures its meaning. This is what makes semantic search possible later — similar meanings end up with similar numbers.
- `from langchain_text_splitters import RecursiveCharacterTextSplitter` — Long articles are too big to feed to an AI model or search efficiently in one piece, so this tool chops them into smaller, overlap-aware chunks.

Line-by-line:

- `load_dotenv()` — Actually loads whatever is in the `.env` file into the program’s environment so later code (like the embeddings model or an API call) can find any keys it needs.
- `urls = [...]` — A plain list of three blog post links. These are the three source articles our chatbot will “know about” — they’re about AI agents, prompt engineering, and adversarial attacks against LLMs. This matches what the router prompt later says the vectorstore contains.
- `docs = [WebBaseLoader(url).load() for url in urls]` — For every URL in the list, open it and download/parse its text. This produces one list per URL (a list of lists).
- `docs_list = [item for sublist in docs for item in sublist]` — Flattens that list of lists into a single flat list of documents, so we don’t have to deal with nested lists later.
- `text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(chunk_size=250, chunk_overlap=0)` — Sets up the chunk-cutting tool. `chunk_size=250` means each chunk will be about 250 tokens (roughly small paragraph-sized pieces). `chunk_overlap=0` means chunks don’t repeat any text between them — each chunk is a clean, separate slice.
- `doc_splits = text_splitter.split_documents(docs_list)` — Actually applies the chunk-cutting to all three downloaded articles, producing many small chunks instead of 3 huge documents.
- `embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")` — Loads a specific, well-known small embedding model that will convert each text chunk into a vector of numbers representing its meaning.
- `vectorstore = Chroma.from_documents(...)` — This is the “build the database” step. It takes every chunk, runs it through the embedding model, and stores the resulting vectors (plus the original text) inside a Chroma database. `collection_name="rag-chroma"` names this specific dataset inside Chroma (so you could have multiple separate collections). `persist_directory="./.chroma"` tells Chroma to save the database to disk in a folder called `.chroma`, so it doesn’t have to be rebuilt every time you run the program.
- `retriever = Chroma(...).as_retriever()` — This re-opens the same saved database (using the same collection name and folder) and wraps it as a “retriever” object — something with a simple `.invoke(question)` method that, given a question, returns the most relevant chunks. This `retriever` variable is what gets imported and used later in `graph/nodes/retrieve.py`.

```

from typing import List, TypedDict

class GraphState(TypedDict):
    """
    Represent the state of our graph.

    Attributes:
        question: question
        generation: LLM generation
        web_search: whether to add search
        documents: list of documents
    """

    question: str
    generation: str
    web_search: bool
    documents: List[str]

```

Imports:

- `from typing import List, TypedDict` — `TypedDict` lets us define a dictionary where we specify in advance exactly which keys it must have and what type each value should be. It's basically a way to say "this dictionary always looks like this" so tools (and other programmers) can catch mistakes early. `List` is just used to describe that `documents` will be a list.

Line-by-line:

- `class GraphState(TypedDict):` — Defines a new dictionary "shape" called `GraphState`. Every node in the graph reads from and writes to a dictionary of this shape — think of it as a shared clipboard that gets passed from worker to worker.
- The docstring (the text in triple-quotes) is just a comment explaining what each field means — it doesn't run as code, it's there for humans reading the code.
- `question: str` — The field that holds the user's original question, as plain text.
- `generation: str` — The field that will eventually hold the LLM's generated answer.
- `web_search: bool` — A true/false flag. True means "we need to fall back to a web search," set by the document-grading step.
- `documents: List[str]` — A list holding the retrieved (and later possibly web-searched) document chunks that back up the answer.

graph/consts.py — Naming Things Once

```

RETRIEVE = "retrieve"
GRADE_DOCUMENTS = "grade_documents"
GENERATE = "generate"
WEBSEARCH = "websearch"

```

No imports needed here — it's just four plain variables. Each one stores the *name* (as text) of a node in the graph. Instead of typing the raw string `"retrieve"` everywhere across many files and risking a typo somewhere, the whole codebase imports `RETRIEVE` from here and refers to it consistently. If you ever wanted to rename a node, you'd change it in exactly one place.

graph/nodes/__init__.py — Making Imports Cleaner

```

from graph.nodes.generate import generate
from graph.nodes.grade_documents import grade_documents
from graph.nodes.retrieve import retrieve
from graph.nodes.web_search import web_search

__all__ = ["generate", "grade_documents", "retrieve", "web_search"]

```

This file exists purely for convenience. Normally you'd have to write `from graph.nodes.retrieve import retrieve`, `from graph.nodes.generate import generate`, etc., separately in `graph.py`. By pulling all four functions into this one `__init__.py` file, `graph.py` can instead do a single line: `from graph.nodes import generate, grade_documents, retrieve, web_search`. The `__all__` list simply documents (and controls) what gets exposed if someone does `from graph.nodes import *`.

graph/nodes/retrieve.py — Fetching Relevant Chunks

```

from typing import Any, Dict

from graph.state import GraphState
from ingestion import retriever

def retrieve(state: GraphState) -> Dict[str, Any]:

```

```

print("====RETRIEVE====")
question = state["question"]

documents = retriever.invoke(question)

return {"documents": documents, "question": question}

```

Imports:

- `from typing import Any, Dict` — Used only to describe the function’s return type (a dictionary whose values can be of any type). This is a “type hint” — it doesn’t change how the code runs, it just helps humans and editors understand what’s being returned.
- `from graph.state import GraphState` — Brings in the shared-notebook shape we defined earlier, so this function’s input can be labeled with that type.
- `from ingestion import retriever` — This is the important one: it imports the actual `retriever` object we built in `ingestion.py` (the one connected to our Chroma vector database). Note: importing this file also re-runs everything in `ingestion.py` — including re-downloading and re-embedding the 3 articles — unless that step has been separated out, so this coupling is worth knowing about.

Line-by-line:

- `def retrieve(state: GraphState) -> Dict[str, Any]:` — Defines the node function. It receives the current shared state and must return a dictionary of updates to merge back into that state.
- `print("====RETRIEVE====")` — Just a visual marker printed to the terminal so that when you run the program, you can see which step is currently executing. Purely for debugging/readability.
- `question = state["question"]` — Pulls the user’s question out of the shared state.
- `documents = retriever.invoke(question)` — Sends the question to the vector database retriever, which returns the most semantically similar chunks of text from our stored articles.
- `return {"documents": documents, "question": question}` — Hands back the retrieved documents (and the question, unchanged) so LangGraph can merge these into the shared state for the next node to use.

graph/nodes/grade_documents.py — Filtering Out Irrelevant Chunks

```

from typing import Any, Dict

from graph.chains.retrieval_grader import retrieval_grader
from graph.state import GraphState


def grade_documents(state: GraphState) -> Dict[str, Any]:
    """
    Determines whether the retrieved documents are relevant to the question
    If documents is not relevant, we will return a flag to run web search

    Args:
        state (dict): The current graph state

    Returns:
        state (dict): Filtered out irrelevant documents and updated web_search state
    """

    print("====CHECK DOCUMENT RELEVANCE TO QUESTION====")
    question = state["question"]
    documents = state["documents"]

    filtered_docs = []
    web_search = False
    for d in documents:
        score = retrieval_grader.invoke(
            {"question": question, "document": d.page_content}
        )
        grade = score.binary_score
        if grade.lower() == "yes":
            print("====GRADE: DOCUMENT RELEVANT====")
            filtered_docs.append(d)
        else:
            print("====GRADE: DOCUMENT NOT RELEVANT====")
            web_search = True
            continue
    return {"documents": filtered_docs, "question": question, "web_search": web_search}

```

Imports:

- `from typing import Any, Dict` — Same reason as before, just for type hints on the return value.
- `from graph.chains.retrieval_grader import retrieval_grader` — Brings in the pre-built LLM chain whose only job is: “given a question and one document, say yes or no — is it relevant?” This node uses that chain to actually do the grading.

- `from graph.state import GraphState` — Same shared-notebook type as before.

Line-by-line:

- `def grade_documents(state: GraphState) -> Dict[str, Any]:` — Defines the node. The docstring below it explains in plain English what the function does — it's a comment block, not executable code.
- `print("===CHECK DOCUMENT RELEVANCE TO QUESTION===")` — Debug marker, same idea as before.
- `question = state["question"] / documents = state["documents"]` — Pulls the question and the list of retrieved documents out of the shared state.
- `filtered_docs = []` — An empty list that will collect only the documents judged relevant.
- `web_search = False` — Starts as False; will flip to True the moment even one document is judged irrelevant.
- `for d in documents:` — Loops through every retrieved document chunk, one at a time.
- `score = retrieval_grader.invoke({"question": question, "document": d.page_content})` — Calls the LLM-based grader chain, giving it the question and the raw text of this one chunk (`d.page_content`), and gets back a structured score object.
- `grade = score.binary_score` — Pulls out the actual “yes”/“no” text field from that score object.
- `if grade.lower() == "yes":` — Lower-cases the grade just in case the LLM returns “Yes” or “YES” instead of exactly “yes”, then checks if it's a yes.
- `filtered_docs.append(d)` — If relevant, keep this document in our filtered list.
- `else: ... web_search = True; continue` — If not relevant, flip the flag to True (meaning: we might not have enough good material, so we may need to search the web later) and skip to the next document without adding this one.
- `return {...}` — Sends back the filtered documents, the unchanged question, and the (possibly True) `web_search` flag so the graph can decide what to do next.

graph/nodes/generate.py — Writing the Answer

```
from typing import Any, Dict

from graph.chains.generation import generation_chain
from graph.state import GraphState

def generate(state: GraphState) -> Dict[str, Any]:
    print("---GENERATE---")
    question = state["question"]
    documents = state["documents"]

    generation = generation_chain.invoke({"context": documents, "question": question})
    return {"documents": documents, "question": question, "generation": generation}
```

Imports:

- `from typing import Any, Dict` — Type hints again.
- `from graph.chains.generation import generation_chain` — Brings in the actual “write an answer” LLM chain (prompt + model + output formatter).
- `from graph.state import GraphState` — Shared state type.

Line-by-line:

- `def generate(state: GraphState) -> Dict[str, Any]:` — Defines the node.
- `print("---GENERATE---")` — Debug marker.
- `question = state["question"] / documents = state["documents"]` — Grabs the question and the (already filtered / possibly web-augmented) documents from the state.
- `generation = generation_chain.invoke({"context": documents, "question": question})` — Feeds both into the generation chain — the documents become the “context” the LLM must base its answer on, and the question is what it must answer. Returns the LLM’s written answer as plain text.
- `return {...}` — Passes the documents, question, and the new `generation` (the answer text) back into the shared state.

graph/nodes/web_search.py — Falling Back to the Internet

```
from typing import Any, Dict

from dotenv import load_dotenv
from langchain_core.documents import Document
from langchain_tavily import TavilySearch

from graph.state import GraphState

load_dotenv()

web_search_tool = TavilySearch(max_result=3)
```

```
def web_search(state: GraphState) -> Dict[str, Any]:
    print("====WEB SEARCH====")
    question = state["question"]
    documents = state["documents"]

    tavily_results = web_search_tool.invoke({"query": question})

    joined_tavily_result = "\n".join(
        [tavily_result["content"] for tavily_result in tavily_results]
    )

    web_results = Document(page_content=joined_tavily_result)
    if documents is not None:
        documents.append(web_results)
    else:
        documents = [web_results]
    return {"documents": documents, "question": question}

if __name__ == "__main__":
    web_search(state={"question": "agent memory", "documents": None})
```

Imports — why each one is here:

- `from typing import Any, Dict` — Type hints, same as always.
- `from dotenv import load_dotenv` — So this file can also load API keys (in this case, the Tavily search API key) from the `.env` file, in case this file gets run on its own.
- `from langchain_core.documents import Document` — This is LangChain’s standard “wrapper” for a chunk of text. All documents flowing through the graph — whether they came from our vector database or a web search — are expected to be `Document` objects, so web search results need to be wrapped in one to stay consistent.
- `from langchain_tavily import TavilySearch` — Tavily is a search engine API built specifically for AI agents. This import gives us a ready-made tool to call it.
- `from graph.state import GraphState` — Shared state type.

Line-by-line:

- `load_dotenv()` — Loads environment variables (like the `TAVILY_API_KEY`) so the search tool below can authenticate.
- `web_search_tool = TavilySearch(max_result=3)` — Creates one instance of the Tavily search tool, configured to return at most 3 results per search. This is created once, outside the function, so it isn’t rebuilt every single time the node runs.
- `def web_search(state: GraphState) -> Dict[str, Any]:` — Defines the node.
- `print("====WEB SEARCH====")` — Debug marker.
- `question = state["question"] / documents = state["documents"]` — Grabs the current question and whatever documents already exist in state (could be `None` or a list from a previous retrieval).
- `tavily_results = web_search_tool.invoke({"query": question})` — Actually performs the web search using the question as the search query, returning a list of result dictionaries.
- `joined_tavily_result = "\n".join([tavily_result["content"] for tavily_result in tavily_results])` — Goes through each search result, pulls out just its `"content"` text, and glues all of them together into one big block of text, separated by newlines.
- `web_results = Document(page_content=joined_tavily_result)` — Wraps that combined web-search text into a single `Document` object, matching the format the rest of the pipeline expects.
- `if documents is not None: documents.append(web_results)` — If we already had some documents (e.g., some survived the relevance grading), just add the new web result onto that existing list.
- `else: documents = [web_results]` — If there were no documents at all yet, start a brand-new list containing just this web result.
- `return {"documents": documents, "question": question}` — Sends the updated documents list (now including the web search result) back into the shared state.
- `if __name__ == "__main__": web_search(state={...})` — This block only runs if you execute this specific file directly (`python web_search.py`), not when it’s imported by another file. It’s a small manual test that calls the function with a sample question, useful for debugging this node in isolation.

graph/chains/retrieval_grader.py — “Is This Chunk Relevant?”

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_groq import ChatGroq
from pydantic import BaseModel, Field

llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0)

class GradeDocuments(BaseModel):
    """Binary score for relevance check on retrieved documents"""
```

```

    binary_score: str = Field(
        description="Documents are relevant to the question, 'yes' or 'no'"
    )

structured_llm_grader = llm.with_structured_output(GradeDocuments)

system = """You are a grader assessing relevance of a retrieved document to a user question.
If the document contains keyword(s) or semantic meaning related to the question, grade it as relevant.
Give a binary score 'yes' or 'no' to indicate whether the document is relevant to the question."""

grade_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system),
        ("human", "Retrieved document: \n\n {document} \n\n User question: {question}"),
    ]
)

retrieval_grader = grade_prompt | structured_llm_grader

```

Imports:

- `from langchain_core.prompts import ChatPromptTemplate` — A tool for building reusable message templates with blanks (like `{question}`) that get filled in later — instead of manually gluing strings together every time.
- `from langchain_groq import ChatGroq` — Groq is a very fast AI inference provider. This import lets us call an LLM hosted on Groq's servers.
- `from pydantic import BaseModel, Field` — Pydantic lets us define a strict “shape” for data (similar spirit to `TypedDict`, but with runtime validation). Here it's used to force the LLM's output into a clean, predictable structure instead of free-form text we'd have to parse ourselves.

Line-by-line:

- `llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0)` — Creates a connection to a specific LLM model hosted on Groq. `temperature=0` means “be as deterministic and non-random as possible” — good for a yes/no grading task where you want consistent judgments, not creativity.
- `class GradeDocuments(BaseModel):` — Defines the exact shape we want the LLM's answer to come back in.
- `binary_score: str = Field(description="...")` — Declares one field, `binary_score`, that must be a string, and gives a plain-English description of what it should contain (“yes” or “no”). This description is actually shown to the LLM behind the scenes to guide its output.
- `structured_llm_grader = llm.with_structured_output(GradeDocuments)` — Wraps the LLM so that instead of returning free-form text, it's forced to return data matching the `GradeDocuments` shape (an object with a `.binary_score` field) — no manual text-parsing needed.
- `system = """..."""` — The system instructions given to the LLM, explaining its role: act as a grader, and decide relevance based on keyword or meaning match.
- `grade_prompt = ChatPromptTemplate.from_messages([...])` — Builds the full prompt template: a system message (the instructions above) plus a human message with two blanks, `{document}` and `{question}`, which get filled in each time this chain is called.
- `retrieval_grader = grade_prompt | structured_llm_grader` — The `|` (pipe) here chains the prompt and the LLM together: first fill in the prompt template, then send it to the structured LLM. The result, `retrieval_grader`, is now a single callable object — call `.invoke([...])` on it and it runs the whole pipeline in one step. This is what `grade_documents.py` imports and uses.

graph/chains/generation.py — “Write the Final Answer”

```

from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_groq import ChatGroq

llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0)
prompt = ChatPromptTemplate.from_messages(
    [
        (
            "human",
            "You are an assistant for question-answering tasks. "
            "Use the following pieces of retrieved context to answer the question. "
            "If you don't know the answer, just say that you don't know.\n"
            "Use three sentences maximum and keep the answer concise.\n"
            "Question: {question} \nContext: {context} \nAnswer:",
        )
    ]
)

```

```
generation_chain = prompt | llm | StrOutputParser()
```

Imports:

- `from langchain_core.output_parsers import StrOutputParser` — This tool takes whatever object the LLM technically returns and extracts just the plain text string out of it, so the rest of the code can work with simple text instead of a wrapped object.
- `from langchain_core.prompts import ChatPromptTemplate` — Same templating tool as before, for building the prompt with blanks.
- `from langchain_groq import ChatGroq` — Same Groq LLM connector as before.

Line-by-line:

- `llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0)` — Same model, same “be consistent, not creative” setting — sensible for factual Q&A grounded in specific documents.
- `prompt = ChatPromptTemplate.from_messages([("human", "...")])` — Builds a single-message prompt (just a “human” role message, no separate system message here) that gives the LLM its instructions and two blanks: `{question}` and `{context}`. The instructions tell it to answer using only the given context, admit when it doesn’t know, and keep the answer to 3 sentences max — this keeps answers short and grounded instead of long-winded or made up.
- `generation_chain = prompt | llm | StrOutputParser()` — Chains three steps together: fill in the prompt → send to the LLM → strip out just the plain text answer. Calling `.invoke({"context": ..., "question": ...})` on this runs the whole thing and hands back a clean string, which is what `generate.py` uses.

graph/chains/hallucination_grader.py — “Is the Answer Actually Grounded in the Facts?”

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableSequence
from langchain_groq import ChatGroq
from pydantic import BaseModel, Field

llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0)

class GradeHallucinations(BaseModel):
    """Binary score for hallucination present in generation answer."""

    binary_score: bool = Field(
        description="Answer is grounded in the facts, 'yes' or 'no'"
    )

structured_llm_grader = llm.with_structured_output(GradeHallucinations)

system = """You are a grader assessing whether an LLM generation is grounded in / supported by a set of retrieved facts.
Give a binary score 'yes' or 'no'. 'Yes' means that the answer is grounded in / supported by the facts."""

hallucination_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system),
        (
            "human",
            "Set of facts: \n\n {documents} \n\n LLM generation: {generation}",
        ),
    ],
)

hallucination_grader: RunnableSequence = hallucination_prompt | structured_llm_grader
```

Imports:

- `from langchain_core.prompts import ChatPromptTemplate` — Same prompt-templating tool.
- `from langchain_core.runnables import RunnableSequence` — This is just the “type name” for a chained pipeline (the result of using `|` between components). It’s imported here purely to label the final `hallucination_grader` variable with its correct type for clarity — it doesn’t change behavior.
- `from langchain_groq import ChatGroq` — Same LLM connector.
- `from pydantic import BaseModel, Field` — Same structured-output tool as in the retrieval grader.

Line-by-line:

- `llm = ChatGroq(...)` — Same model setup as the other chains.
- `class GradeHallucinations(BaseModel):` — Defines the expected shape of this grader’s output.
- `binary_score: bool = Field(description="...")` — Note this one is a `bool` (True/False) rather than a raw string like the retrieval

grader — a small inconsistency in the codebase, but both work fine since LangChain will coerce the LLM’s structured output into whichever type you declare.

- `structured_llm_grader = llm.with_structured_output(GradeHallucinations)` — Forces the LLM’s response into that True/False shape.
- `system = """..."""` — Instructs the LLM to act as a fact-checker: compare the generated answer against the actual retrieved documents and say whether it’s genuinely supported by them.
- `hallucination_prompt = ChatPromptTemplate.from_messages([...])` — Builds the full prompt with two blanks: `{documents}` (the facts) and `{generation}` (the answer to be checked).
- `hallucination_grader: RunnableSequence = hallucination_prompt | structured_llm_grader` — Chains the prompt and the structured LLM together into one callable pipeline. This is what `graph.py` calls right after an answer is generated, to catch made-up (hallucinated) content before it reaches the user.

`graph/chains/answer_grader.py` — “Does the Answer Actually Address the Question?”

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableSequence
from langchain_groq import ChatGroq
from pydantic import BaseModel, Field

class GradeAnswer(BaseModel):
    """
    A grader that checks if an answer addresses the question.
    """
    binary_score: bool = Field(
        description="Answer addresses the question, 'yes' or 'no'"
    )

llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0)

structured_llm_grader = llm.with_structured_output(GradeAnswer)

system = """You are a grader assessing whether an answer addresses / resolves a question.
Give a binary score 'yes' or 'no'. Yes means that the answer resolves the question."""

answer_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system),
        (
            "human",
            "User question: \n\n {question} \n\n LLM generation: {generation}",
        ),
    ],
)

answer_grader: RunnableSequence = answer_prompt | structured_llm_grader
```

This file is structurally almost identical to the hallucination grader, just checking a different thing.

Imports: Same four as the hallucination grader, for the same reasons — prompt templating, the type label for the final pipeline, the Groq LLM connection, and Pydantic for structured output.

Line-by-line:

- `class GradeAnswer(BaseModel): binary_score: bool = Field(...)` — Defines a True/False result meaning “does this answer actually resolve the question the user asked?” This is a different check from the hallucination grader: an answer could be 100% grounded in real facts, yet still fail to actually answer what was asked — this chain catches that specific case.
- `llm = ChatGroq(...)` — Same model configuration as the other chains.
- `structured_llm_grader = llm.with_structured_output(GradeAnswer)` — Forces output into the True/False shape.
- `system = """..."""` — Tells the LLM its job: judge whether the given answer resolves the given question.
- `answer_prompt = ChatPromptTemplate.from_messages([...])` — Builds the prompt with two blanks: `{question}` and `{generation}` (the answer to be judged).
- `answer_grader: RunnableSequence = answer_prompt | structured_llm_grader` — Chains everything into one callable pipeline, used by `graph.py` as the second and final quality check on a generated answer.

`graph/chains/router.py` — “Which Path Should This Question Take?”

```
from typing import Literal

from langchain_core.prompts import ChatPromptTemplate
```

```

from pydantic import BaseModel, Field
from langchain_groq import ChatGroq

class RouteQuery(BaseModel):
    """Route a user query to the most relevant datasource."""

    datasource: Literal["vectorstore", "websearch"] = Field(
        default="...",
        description="Given a user question choose to route it to web search or a vectorstore.",
    )

llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0)
structured_llm_router = llm.with_structured_output(RouteQuery)

system = """You are an expert at routing a user question to a vectorstore or web search.
The vectorstore contains documents related to agents, prompt engineering, and adversarial attacks.
Use the vectorstore for questions on these topics. For all else, use web-search."""

route_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system),
        ("human", "{question}"),
    ]
)

question_router = route_prompt | structured_llm_router

```

Imports:

- `from typing import Literal` — `Literal` lets us restrict a value to only a specific, fixed set of exact options — here, the router's decision can only ever be exactly `"vectorstore"` or exactly `"websearch"`, nothing else. This prevents the LLM's structured output from drifting into some unexpected third value.
- `from langchain_core.prompts import ChatPromptTemplate` — Same prompt-templating tool as everywhere else.
- `from pydantic import BaseModel, Field` — Same structured-output tool.
- `from langchain_groq import ChatGroq` — Same LLM connector.

Line-by-line:

- `class RouteQuery(BaseModel):` — Defines the shape of the router's decision.
- `datasource: Literal["vectorstore", "websearch"] = Field(default="...", description="...")` — The one field this class holds: which path to take. It must be exactly one of the two allowed strings. The `default="..."` is just a placeholder default value (it will normally be overwritten by whatever the LLM actually decides).
- `llm = ChatGroq(...)` — Same model setup as the other chains.
- `structured_llm_router = llm.with_structured_output(RouteQuery)` — Forces the LLM's decision into that strict `RouteQuery` shape.
- `system = """..."""` — Tells the LLM exactly what topics the vectorstore covers (agents, prompt engineering, adversarial attacks) — matching the three articles we actually embedded in `ingestion.py` — and says everything else should go to web search.
- `route_prompt = ChatPromptTemplate.from_messages([("system", system), ("human", "{question}")])` — Builds the prompt: the system instructions plus the raw user question dropped straight in as the human message.
- `question_router = route_prompt | structured_llm_router` — Chains it all into one callable pipeline. This is the very first thing that runs in `graph.py` when a new question comes in — it decides whether to go retrieve from the vectorstore or jump straight to a web search.

graph/graph.py — Wiring Everything Into a Flowchart

```

from dotenv import load_dotenv
from langgraph.graph import END, StateGraph

from graph.chains.answer_grader import answer_grader
from graph.chains.hallucination_grader import hallucination_grader
from graph.consts import GENERATE, GRADE_DOCUMENTS, RETRIEVE, WEBSEARCH
from graph.nodes import generate, grade_documents, retrieve, web_search
from graph.state import GraphState
from graph.chains.router import question_router, RouteQuery

load_dotenv()

```

Imports:

- `from dotenv import load_dotenv` — Same purpose as before: load API keys from `.env`.

- `from langgraph.graph import END, StateGraph` — `StateGraph` is the core LangGraph class used to actually build the flowchart of nodes and edges. `END` is a special built-in marker meaning “the graph is finished, stop here.”
- `from graph.chains.answer_grader import answer_grader` and `from graph.chains.hallucination_grader import hallucination_grader` — Brings in the two quality-check chains this file will use directly (rather than through a node file, since these checks are short enough to run inline inside decision functions here).
- `from graph.consts import GENERATE, GRADE_DOCUMENTS, RETRIEVE, WEBSEARCH` — Brings in the constant names for each node, so the wiring below reads as `RETRIEVE` instead of the raw string `"retrieve"`.
- `from graph.nodes import generate, grade_documents, retrieve, web_search` — Brings in the four actual worker functions to attach to the graph.
- `from graph.state import GraphState` — The shared state shape, used to type-hint the graph.
- `from graph.chains.router import question_router, RouteQuery` — Brings in the routing chain and its result-shape class, used to decide where a new question should start.
- `load_dotenv()` — Loads environment variables again (this file can also be run somewhat independently, so it’s safe to call again).

```
def decide_to_generate(state):
    print("---ASSESS GRADED DOCUMENTS---")

    if state["web_search"]:
        print(
            "---DECISION: NOT ALL DOCUMENTS ARE NOT RELEVANT TO QUESTION, INCLUDE WEB SEARCH---"
        )
        return WEBSEARCH

    else:
        print("---DECISION: GENERATE---")
        return GENERATE
```

- `def decide_to_generate(state):` — A decision function (not a worker node) — its whole job is to look at the state and return the *name* of whichever node should run next.
- `if state["web_search"]:` — Checks the flag that `grade_documents` set earlier. If it’s True (meaning at least one retrieved document was irrelevant), we decide we need a web search.
- `return WEBSEARCH` — Tells the graph: go to the web search node next.
- `else: return GENERATE` — Otherwise, we had enough good documents already, so go straight to generating the answer.
- (The print statement in the “if” branch has a small wording slip — “NOT ALL DOCUMENTS ARE NOT RELEVANT” — this is just a debug message and doesn’t affect the logic, only what gets printed to the terminal.)

```
def grade_generation_grounded_in_documents_and_question(state: GraphState,) -> str:
    print("---CHECK HALLUCINATIONS---")
    question = state["question"]
    documents = state["documents"]
    generation = state["generation"]

    score = hallucination_grader.invoke(
        {"documents": documents, "generation": generation}
    )

    if hallucination_grade := score.binary_score:
        print("---DECISION: GENERATION IS GROUNDED IN DOCUMENTS---")
        print("---GRADE GENERATION VS QUESTION---")

        score = answer_grader.invoke(
            {"question": question, "generation": generation}
        )

        if answer_grade := score.binary_score:
            print("---DECISION: GENERATION ADDRESSES QUESTION---")
            return "useful"
        else:
            print("---DECISION: GENERATION DOES NOT ADDRESS QUESTION---")
            return "not useful"

    else:
        print("---DECISION: GENERATION IS NOT GROUNDED IN DOCUMENTS, RE-TRY---")
        return "not supported"
```

- `def grade_generation_grounded_in_documents_and_question(state: GraphState,) -> str:` — Another decision function, this time the most important one: it decides what happens right after an answer has been generated.
- `question = state["question"], documents = state["documents"], generation = state["generation"]` — Pulls out everything needed: the original question, the facts used, and the answer that was written.
- `score = hallucination_grader.invoke({"documents": documents, "generation": generation})` — Runs the hallucination-check chain: is the generated answer actually backed up by the documents?

- `if hallucination_grade := score.binary_score:` — This uses Python’s “walrus operator” (`:=`), which lets you assign a variable *and* use it in the same line. In plain terms: “save the result into `hallucination_grade`, and if that result is truthy (True), enter this branch.”
- Inside that branch (answer IS grounded in facts): it then runs a **second** check — `answer_grader.invoke(...)` — asking: does this grounded answer actually address the original question?
- `if answer_grade := score.binary_score:` — Same walrus-operator pattern for this second check.
- `return "useful"` — Both checks passed: the answer is grounded AND on-topic. This is the “good” outcome — the graph will stop here and show the answer.
- `return "not useful"` — The answer was grounded in facts, but didn’t actually answer the question. The graph will fall back to a web search and try again with fresh material.
- `else: return "not supported"` — The very first check failed: the answer wasn’t grounded in the documents at all (possible hallucination). The graph will send the flow back to generate again (perhaps hoping for a better result, though note it’s using the same documents, so this specific retry logic has a known limitation — it doesn’t necessarily fetch new material before retrying).

```
def route_question(state: GraphState) -> str:
    print("---ROUTE QUESTION---")
    question = state["question"]

    source: RouteQuery = question_router.invoke({"question": question})

    if source.datasource == WEBSEARCH:
        print("---ROUTE QUESTION TO WEB SEARCH---")
        return WEBSEARCH

    elif source.datasource == "vectorstore":
        print("---ROUTE QUESTION TO RAG---")
        return RETRIEVE
```

- `def route_question(state: GraphState) -> str:` — The very first decision made in the whole graph, for a brand-new incoming question.
- `question = state["question"]` — Grabs the question.
- `source: RouteQuery = question_router.invoke({"question": question})` — Calls the router chain, which returns a `RouteQuery` object telling us whether this question belongs in `"vectorstore"` or `"websearch"`.
- `if source.datasource == WEBSEARCH: return WEBSEARCH` — If the router said web search, go straight there (skip retrieval entirely).
- `elif source.datasource == "vectorstore": return RETRIEVE` — If the router said vectorstore, start with the retrieve node instead. (Notice this compares against the raw string `"vectorstore"` rather than a constant — a minor inconsistency, since there’s no `VECTORSTORE` constant defined in `consts.py`, but it works fine because `RouteQuery`’s `Literal` type guarantees it can only ever be exactly that string.)

```
workflow = StateGraph(GraphState)

workflow.add_node(RETRIEVE, retrieve)
workflow.add_node(GRADE_DOCUMENTS, grade_documents)
workflow.add_node(GENERATE, generate)
workflow.add_node(WEBSEARCH, web_search)
```

- `workflow = StateGraph(GraphState)` — Creates a brand-new, empty graph, telling it that every piece of state flowing through it must follow the `GraphState` shape.
- `workflow.add_node(RETRIEVE, retrieve)` — Registers the `retrieve` function as a node in the graph, and gives it the name `RETRIEVE` (which equals `"retrieve"`) so it can be referred to by that name in edges.
- The next three lines do the exact same thing for the `grade_documents`, `generate`, and `web_search` functions — registering each as a named node.

```
workflow.set_conditional_entry_point(
    route_question,
    path_map={
        WEBSEARCH: WEBSEARCH,
        RETRIEVE: RETRIEVE,
    },
)
```

- `workflow.set_conditional_entry_point(route_question, path_map={...})` — This sets up the *starting point* of the graph, but instead of always starting at the same node, it says “run the `route_question` function first, and whatever name it returns, go to that node.” The `path_map` tells LangGraph: if `route_question` returns the value `WEBSEARCH`, actually jump to the node named `WEBSEARCH`; if it returns `RETRIEVE`, jump to the node named `RETRIEVE`.

```
workflow.add_edge(RETRIEVE, GRADE_DOCUMENTS)
workflow.add_conditional_edges(
    GRADE_DOCUMENTS,
    decide_to_generate,
    path_map={
```

```

        WEBSEARCH: WEBSEARCH,
        GENERATE: GENERATE,
    },
)

```

- `workflow.add_edge(RETRIEVE, GRADE_DOCUMENTS)` — A fixed (unconditional) arrow: after `retrieve` finishes, always go straight to `grade_documents` next.
- `workflow.add_conditional_edges(GRADE_DOCUMENTS, decide_to_generate, path_map={...})` — After `grade_documents` finishes, don't go to a fixed next step — instead run the `decide_to_generate` function on the current state, and whatever it returns (`WEBSEARCH` or `GENERATE`), go to that node.

```

workflow.add_conditional_edges(
    GENERATE,
    grade_generation_grounded_in_documents_and_question,
    path_map={
        "not supported": GENERATE,
        "useful": END,
        "not useful": WEBSEARCH,
    },
)

```

- Same pattern again, but after the `generate` node: run `grade_generation_grounded_in_documents_and_question` on the current state, and depending on its return value, either loop back to `generate` again (`"not supported"`), finish the whole graph successfully (`"useful" → END`), or fall back to a web search (`"not useful"`).

```

workflow.add_edge(WEBSEARCH, GENERATE)
workflow.add_edge(GENERATE, END)

app = workflow.compile()

app.get_graph().draw_mermaid_png(output_file_path="graph.png")

```

- `workflow.add_edge(WEBSEARCH, GENERATE)` — Fixed arrow: whenever the web search node runs, always go to generate next afterward.
- `workflow.add_edge(GENERATE, END)` — A plain fallback fixed edge from generate to the end. (In practice, the conditional edges added just above already decide what happens after `generate`, so this line mostly acts as a safety default within LangGraph's internal bookkeeping.)
- `app = workflow.compile()` — “Compiles” the graph — locks in all the nodes and edges you've defined and turns it into a runnable object called `app`, which is what actually gets executed when you call `.invoke(...)`.
- `app.get_graph().draw_mermaid_png(output_file_path="graph.png")` — Automatically generates a visual picture of the flowchart you just built (using the Mermaid diagram format under the hood) and saves it as `graph.png`. This is why the repo already contains a `graph.png` file — it gets (re)created every time this file runs.

main.py — Actually Running It

```

from dotenv import load_dotenv

load_dotenv()

from graph.graph import app

if __name__ == "__main__":
    print("Hello Advanced RAG")
    print(app.invoke(input={"question": "what is agent memory?"}))

```

Imports:

- `from dotenv import load_dotenv` — Same environment-variable loader as before.
- `from graph.graph import app` — Imports the fully compiled, ready-to-run graph object we built in `graph.py`. (This import is deliberately placed *after* `load_dotenv()` runs, so that by the time `graph.py` and everything it imports gets loaded — including chains that create LLM connections needing API keys — those keys are already available in the environment.)

Line-by-line:

- `load_dotenv()` — Loads the `.env` file's contents into the environment immediately, before anything else happens.
- `from graph.graph import app` — Now safely import the compiled graph, since keys are already loaded.
- `if __name__ == "__main__":` — Ensures the code below only runs when this file is executed directly (`python main.py`), not when it's imported elsewhere.
- `print("Hello Advanced RAG")` — Just a friendly startup message.
- `print(app.invoke(input={"question": "what is agent memory?"}))` — This is the actual demo run: it sends one hard-coded test question, `"what is agent memory?"`, through the entire graph (route → retrieve/web-search → grade → generate → grade the generation → possibly retry) and prints whatever the final state dictionary looks like once the graph reaches `END`.

Quick Recap: The Full Journey of One Question

1. `main.py` starts the whole thing off by calling `app.invoke({"question": "..."})`.
2. `route_question` (in `graph.py`) uses the `question_router` chain to decide: vectorstore or web search?
3. If vectorstore → `retrieve` node fetches chunks from Chroma (built earlier by `ingestion.py`).
4. `grade_documents` node checks each chunk with `retrieval_grader`, keeps the relevant ones, and flags `web_search=True` if any were bad.
5. `decide_to_generate` checks that flag — if web search is needed, go to `web_search` node (which calls Tavily and appends results); otherwise skip straight to `generate`.
6. `generate` node uses the `generation_chain` to write an answer from whatever documents we now have.
7. `grade_generation_grounded_in_documents_and_question` runs two checks in sequence: `hallucination_grader` (is it grounded in facts?) then `answer_grader` (does it answer the question?).
8. Based on those two checks, the graph either finishes (`END`), retries generation, or falls back to a fresh web search and generates again.

That's the entire system — a small, self-checking pipeline that tries to only give the user answers that are both fact-based and actually useful.